



Faculté des Sciences et Technologies (FST)

Rapport du travail de Laboratoire N° 1_ Modélisation Mathématique

Etudiant : Donsam Jean Gabard NOEL

Professeur : Ismaël SAINT AMOUR

Niveau : L3

Mars 2026

TD 1 : Génération de clés aléatoires

Objectif : Comprendre les ensembles de caractères et l'entropie.

Instructions :

1. Générer des clés de 8, 16 et 32 caractères avec les ensembles :

ascii_lowercase

ascii_letters

digits

ascii_letters + digits + punctuation

2. Calculer l'entropie de chaque clé.

- Génération d'une clé de 8

```
C:\Users\NOEL>python "c:/Users/NOEL/Documents/IUS/TP1-Mod M.py"
1  # Clé de 8 et son entropie
2  import string
3  import random
4  import math
5
6  char_sets = {}
7  "ascii_letters" : string.ascii_letters,
8  "ascii_lowercase" : string.ascii_lowercase,
9  "digits" : string.digits,
10 "ascii_letters + digits + punctuation" : string.ascii_letters+string.digits+string.punctuation
11 }
12
13 def generate_key(char_set, length=8):
14     return ''.join(random.choice(char_set) for _ in range(length))
15
16 def calculate_entropy(key, alphabet=None):
17     if alphabet is None:
18         alphabet = set(key)
19     L = len(alphabet)
20     N = len(key)
21     return N * math.log2(L) if L > 0 else 0
22
23 key_length = 8
24
25 for name, chars in char_sets.items():
26     key = generate_key(chars, key_length)
```

```
C:\Users\NOEL>python "c:/Users/NOEL/Documents/IUS/TP1-Mod M.py"
ascii_letters    yQJeifRM | Entropie = 45.60 bits
ascii_lowercase  utgwmymo | Entropie = 37.60 bits
digits          77331528 | Entropie = 26.58 bits
ascii_letters + digits + punctuation m3@Q%VA' | Entropie = 52.44 bits

C:\Users\NOEL>
```

- Génération d'une clé de 16

```

1 # Clé de 16 et son entropie
2 import string
3 import random
4 import math
5
6 char_sets = {
7     "ascii_letters" : string.ascii_letters,
8     "ascii_lowercase" : string.ascii_lowercase,
9     "digits" : string.digits,
10    "ascii_letters + digits + punctuation" : string.ascii_letters+string.digits+string.punctuation
11 }
12
13 def generate_key(char_set, length=16):
14     return ''.join(random.choice(char_set) for _ in range(length))
15
16 def calculate_entropy(key, alphabet=None):
17     if alphabet is None:
18         alphabet = set(key)
19     L = len(alphabet)
20     N = len(key)
21     return N * math.log2(L) if L > 0 else 0
22

```

```

22
23 key_length = 16
24
25 for name, chars in char_sets.items():
26     key = generate_key(chars, key_length)
27     entropy = calculate_entropy(key, alphabet=set(chars))
28     print(f"{name:15} {key} | Entropie = {entropy:.2f} bits")

```

```

C:\Users\NOEL>python "c:/Users/NOEL/Documents/IUS/TP1-Mod M.py"
ascii_letters   wFqDfpZqilQFQiiw | Entropie = 91.21 bits
ascii_lowercase ksqsoszlewnuhsh | Entropie = 75.21 bits
digits          4852376304988602 | Entropie = 53.15 bits
ascii_letters + digits + punctuation 8"e{.2w3#zP"!xD | Entropie = 104.87 bits

C:\Users\NOEL>

```

- Génération d'une clé de 32

```

C:\Users\NOEL>Documents\IUS> TP1-Mod M.py
1 # Clé de 32 et son entropie
2 import string
3 import random
4 import math
5
6 char_sets = {
7     "ascii_letters" : string.ascii_letters,
8     "ascii_lowercase" : string.ascii_lowercase,
9     "digits" : string.digits,
10    "ascii_letters + digits + punctuation" : string.ascii_letters+string.digits+string.punctuation
11 }
12
13 def generate_key(char_set, length=32):
14     return ''.join(random.choice(char_set) for _ in range(length))
15
16 def calculate_entropy(key, alphabet=None):
17     if alphabet is None:
18         alphabet = set(key)
19     L = len(alphabet)
20     N = len(key)
21     return N * math.log2(L) if L > 0 else 0
22
23 key_length = 32
24
25 for name, chars in char_sets.items():
26     key = generate_key(chars, key_length)

```

```

C:\Users\NOEL>python "c:/Users/NOEL/Documents/IUS/TP1-Mod M.py"
ascii_letters  UAnanqJCjJYwePStgnqXMWngQbwmLLaK | Entropie = 182.41 bits
ascii_lowercase dnloucohsqudsywurmwsncbdlblcofx | Entropie = 150.41 bits
digits         07625780063935111838578227542681 | Entropie = 106.30 bits
ascii_letters + digits + punctuation C)T@KK.vES]!j71RBF>vifhGnNjI}{+o | Entropie = 209.75 bits

C:\Users\NOEL>

```

3. Comparer les entropies et discuter quel type de clé est le plus sécurisé.

La comparaison des entropies montre clairement que la sécurité d'une clé dépend à la fois de la taille de l'alphabet utilisé et de la longueur de la clé. Les clés constituées uniquement de lettres minuscules ou de chiffres offrent une entropie relativement faible, car l'espace de possibilités est limité, ce qui les rend vulnérables aux attaques par force brute. L'ajout des majuscules améliore légèrement la sécurité, mais reste insuffisant pour des systèmes modernes. En revanche, les clés qui combinent lettres, chiffres et ponctuation exploitent un alphabet beaucoup plus riche, ce qui augmente considérablement l'entropie et rend la clé plus difficile à deviner. Ainsi, les clés les plus sécurisées sont celles qui associent un alphabet large et une longueur suffisante, car elles garantissent une meilleure imprévisibilité et une résistance accrue face aux attaques. En pratique, une clé de grande longueur et générée sur un alphabet diversifié est la solution la plus robuste pour assurer la confidentialité et la sécurité des données.

TD 2 : Chiffrement César

Objectif : Comprendre un chiffrement symétrique simple et la rotation des caractères.

Instructions :

1. Écrire une fonction `caesar_cipher(message, shift)` pour chiffrer un message.
2. Écrire la fonction inverse pour déchiffrer.

```

C:\Users\NOEL\Documents\IUS> TP2-ModM.py
1  def caesar_encrypt(message, shift=3):
2      encrypted = ""
3      for c in message:
4          if c.isupper():
5              encrypted += chr((ord(c) - 65 + shift) % 26 + 65)
6          elif c.islower():
7              encrypted += chr((ord(c) - 97 + shift) % 26 + 97)
8          elif c.isdigit():
9              encrypted += chr((ord(c) - 48 + shift) % 10 + 48)
10         else:
11             encrypted += c
12     return encrypted
13
14  def caesar_decrypt(cipher, shift=3):
15      return caesar_encrypt(cipher, -shift)
16
17  message = input("Entrez un message :")
18  shift = int(input("Entrez un chiffre : "))
19  cipher = caesar_encrypt(message, shift)
20  decipher = caesar_decrypt(cipher, shift)
21
22  print("Message original :", message)
23  print("Message chriffre :", cipher)
24  print("Message dechiffre :", decipher)

```

3. Tester sur des messages avec :

- lettres minuscules
- lettres majuscules
- chiffres

```

OUTPUT  DEBUG CONSOLE  PROBLEMS  TERMINAL

Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\Users\NOEL>python c:/Users/NOEL/Documents/IUS/TP2-ModM.py
Entrez un message : bonsoir
Entrez un chiffre : 4
Message original : bonsoir
Message chriffre : fsrwsmv
Message dechiffre : bonsoir

C:\Users\NOEL>python c:/Users/NOEL/Documents/IUS/TP2-ModM.py
Entrez un message : GABARD EST MALADE
Entrez un chiffre : 7
Message original : GABARD EST MALADE
Message chriffre : NHIHYK LZA THSHKL
Message dechiffre : GABARD EST MALADE

C:\Users\NOEL>python c:/Users/NOEL/Documents/IUS/TP2-ModM.py
Entrez un message : 08032004
Entrez un chiffre : 3
Message original : 08032004
Message chriffre : 31365337
Message dechiffre : 08032004

C:\Users\NOEL>

```

TD 3 Chiffrement XOR simple

Objectif : Comprendre le chiffrement par clé avec opérations bit à bit.

Instructions :

1. Générer une clé aléatoire de même longueur que le message en utilisant `string.printable`.
2. Écrire une fonction `xor_cipher(message, key)` pour chiffrer et déchiffrer.
3. Vérifier que le message déchiffré est identique à l'original.

```
C:\Users\NOEL\Documents\IUS\TP3-ModM.PY
1  import string
2  import random
3  import math
4
5  # Fonction XOR
6  def xor_cipher(message, key):
7      return ''.join(chr(ord(m) ^ ord(k)) for m, k in zip(message, key))
8
9  # Saisie du message
10 message = input("Entrez un message : ")
11
12 # Générer une clé aléatoire de même longueur que le message
13 key = ''.join(random.choice(string.printable) for _ in range(len(message)))
14
15 # Chiffrement et déchiffrement
16 cipher = xor_cipher(message, key)
17 decipher = xor_cipher(cipher, key)
18
19 print("Message original :", message)
20 print("Clé :", key)
21 print("Message chiffré :", cipher)
22 print("Message déchiffré :", decipher)
23
```

```
24 # Calcul de l'entropie de la clé
25 def calculate_entropy(key, alphabet=None):
26     if alphabet is None:
27         alphabet = set(key)
28     L = len(alphabet) # taille de l'alphabet
29     N = len(key)      # longueur de la clé
30     return N * math.log2(L) if L > 0 else 0
31
32 entropy = calculate_entropy(key, alphabet=set(string.printable))
33 print(f"Entropie de la clé : {entropy:.2f} bits")
34
```

```

C:\Users\NOEL>python c:/Users/NOEL/Documents/IUS/TP3-ModM.PY
Entrez un message : L'oiseau est dans la niche
Message original : L'oiseau est dans la niche
?}RTHBm%}3h?JP+~@ : <#l,y3Ts
Message chiffré : p♠♥E
V5♠-ZJ&t,#♥V]_ ▼$9H~%
Message déchiffré : L'oiseau est dans la niche
Entropie de la clé : 172.74 bits

C:\Users\NOEL>python c:/Users/NOEL/Documents/IUS/TP3-ModM.PY
Entrez un message : Soldat en approche.
Message original : Soldat en approche.
Clé : c7NfhF|DD4<z0r({}W0@
Message chiffré : 0X"0 2\!*9]
@ G▲?*n
Message déchiffré : Soldat en approche.
Entropie de la clé : 126.23 bits

```

4. Calculer l'entropie de la clé et discuter sa sécurité.

L'analyse de l'entropie de la clé montre que la sécurité du chiffrement XOR dépend directement de la qualité de la clé utilisée. Une clé générée aléatoirement avec un alphabet riche, comme `string.printable`, possède une entropie élevée et rend le message chiffré beaucoup plus difficile à deviner. En revanche, si la clé est courte ou construite sur un alphabet restreint, l'entropie diminue et la sécurité s'affaiblit. Ainsi, bien que le chiffrement XOR soit simple et réversible, sa robustesse repose entièrement sur la longueur et la diversité de la clé. Plus l'entropie est grande, plus la clé est imprévisible et plus le chiffrement est sécurisé face aux attaques par force brute.

TD 4 : Génération et visualisation de l'entropie

Objectif : Relier entropie, longueur de clé et alphabet.

Instructions :

1. Générer des clés de longueurs différentes (4, 8, 16, 32).
2. Pour chaque clé, calculer l'entropie.


```

C: > Users > NOEL > Documents > IUS > TP4-ModM.py
1  import string
2  import random
3  import math
4  import matplotlib.pyplot as plt
5
6  # Fonction pour générer une clé aléatoire
7  def generate_key(charset, length):
8      return ''.join(random.choice(charset) for _ in range(length))
9
10 # Fonction pour calculer l'entropie
11 def calculate_entropy(key, alphabet=None):
12     if alphabet is None:
13         alphabet = set(key)
14     L = len(alphabet) # taille de l'alphabet
15     N = len(key)     # longueur de la clé
16     return N * math.log2(L) if L > 0 else 0
17
18 # Longueurs de clés à tester
19 lengths = [4, 8, 16, 32]
20 alphabet = string.ascii_letters + string.digits + string.punctuation
21 entropies = []
22
23 # Génération des clés et calcul des entropies
24 for length in lengths:
25     key = generate_key(alphabet, length)
26     entropy = calculate_entropy(key, alphabet=set(alphabet))

```

```

22
23 # Génération des clés et calcul des entropies
24 for length in lengths:
25     key = generate_key(alphabet, length)
26     entropy = calculate_entropy(key, alphabet=set(alphabet))
27     entropies.append(entropy)
28     print(f"Longueur={length} | Clé={key} | Entropie={entropy:.2f} bits")
29
30 # Tracer le graphique
31 plt.plot(lengths, entropies, marker='o', linestyle='-', color='b')
32 plt.title("Entropie vs Longueur de clé")
33 plt.xlabel("Longueur de la clé")
34 plt.ylabel("Entropie (bits)")
35 plt.grid(True)
36
37 # Afficher le graphique
38 plt.show()
39

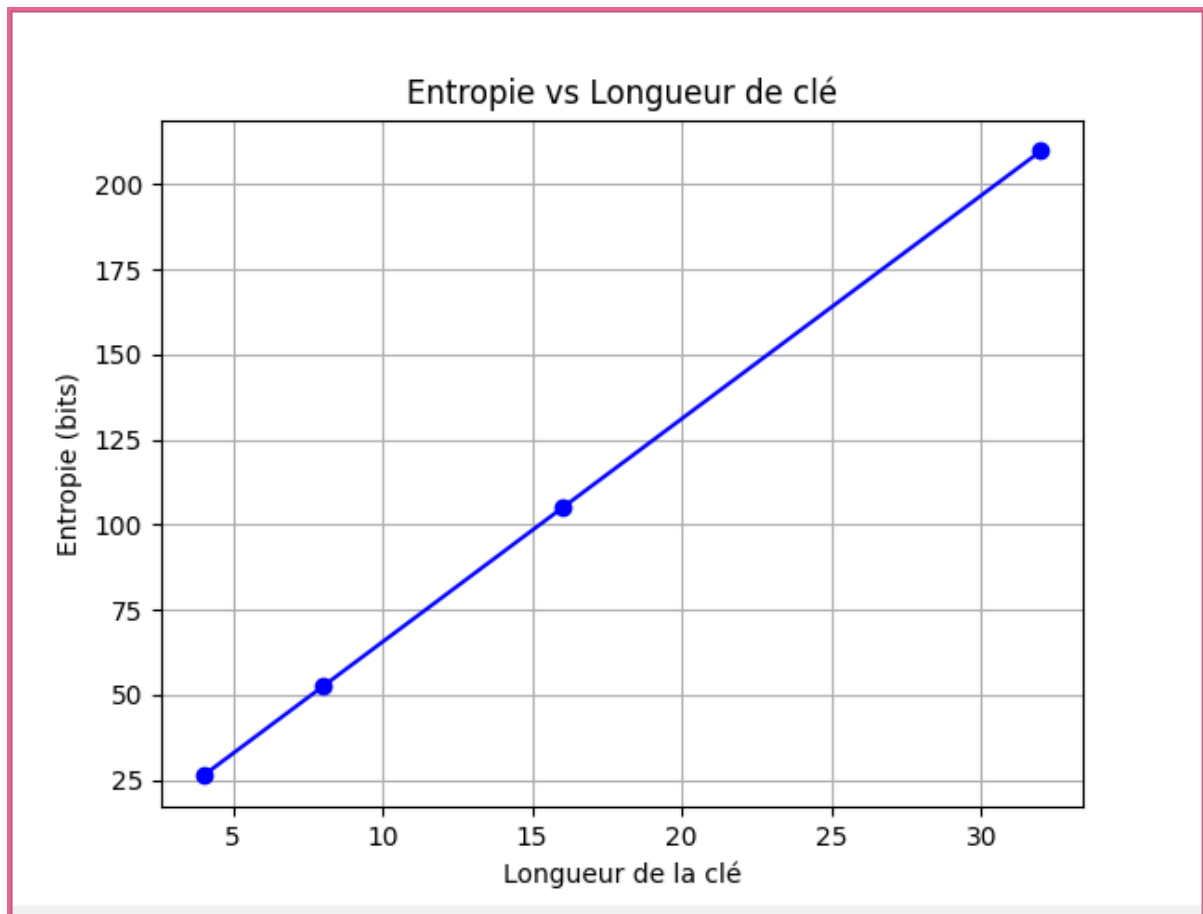
```

```

C:\Users\NOEL>python c:/Users/NOEL/Documents/IUS/TP4-ModM.py
Longueur=4 | Clé=j2zq | Entropie=26.22 bits
Longueur=8 | Clé=oV6/wmD# | Entropie=52.44 bits
Longueur=16 | Clé=<Ni4-140X+|jc|<{ | Entropie=104.87 bits
Longueur=32 | Clé=Rz$E/|N(KW4f|UFj0HR`i'guN}f=b}vQ | Entropie=209.75 bits

```


3. Tracer un graphique entropie vs longueur de clé avec matplotlib.



4. Discuter la relation entre longueur de clé, alphabet et sécurité.

Le graphique entropie vs longueur de clé met en évidence une relation linéaire : plus la clé est longue, plus l'entropie augmente, et donc plus la sécurité est renforcée. De même, l'utilisation d'un alphabet large accroît encore cette entropie, car chaque caractère ajouté offre davantage de possibilités. Une clé courte, même avec un alphabet riche, reste vulnérable, tandis qu'une clé longue avec un alphabet limité ne garantit pas une sécurité optimale. La combinaison idéale est donc une clé suffisamment longue et construite sur un alphabet diversifié, ce qui assure une imprévisibilité maximale et une résistance accrue aux attaques. En pratique, cette relation montre que la sécurité cryptographique repose sur l'équilibre entre la taille de la clé et la richesse de l'alphabet.

Conclusion

En conclusion, ce TD a montré que la sécurité cryptographique repose sur deux piliers essentiels : la longueur de la clé et la diversité de l'alphabet utilisé. Les exercices pratiques, du chiffrement simple (César, XOR) à l'étude de l'entropie, ont permis d'illustrer que plus une clé est longue et construite sur un alphabet riche, plus elle est imprévisible et résistante aux attaques. Ce travail met en évidence l'importance de la modélisation mathématique pour évaluer la robustesse des systèmes et souligne que la combinaison entre taille et variété des caractères est la clé d'une sécurité fiable.

